

C Program - Day-3 Class Assignment

Name: Amit Kumar

Topic: Arrays

Section A: Programs with Output

Question 1: Insert an element at a specified position

Program:

```
/* Q1. Write a C program to insert an element at a specified position. */
#include <stdio.h>

int main()
{
    int arr[100], n, i, element, position;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("Enter the element to insert: ");
    scanf("%d", &element);

    printf("Enter the position (1 to %d): ", n + 1);
    scanf("%d", &position);

    if (position < 1 || position > n + 1)
    {
        printf("Invalid position!\n");
        return 1;
    }

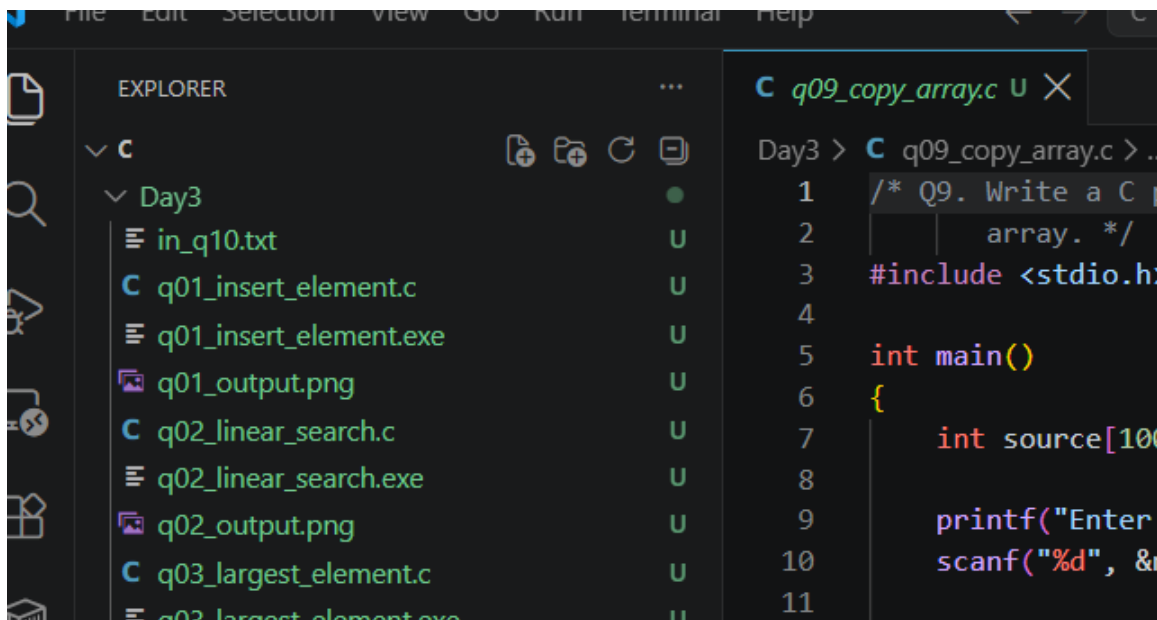
    for (i = n; i >= position; i--)
        arr[i] = arr[i - 1];

    arr[position - 1] = element;
    n++;

    printf("Array after insertion:\n");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}
```

```
    return 0;
}
```

Output:



Question 2: Search for a given value in an array using linear search; print its index or Not found

Program:

```
/* Q2. Search for a given value in an array using linear search;
   print its index or "Not found". */
#include <stdio.h>

int main()
{
    int arr[100], n, i, value;
    int found = 0;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("Enter the value to search: ");
    scanf("%d", &value);

    for (i = 0; i < n; i++)
    {
        if (arr[i] == value)
        {
```

```

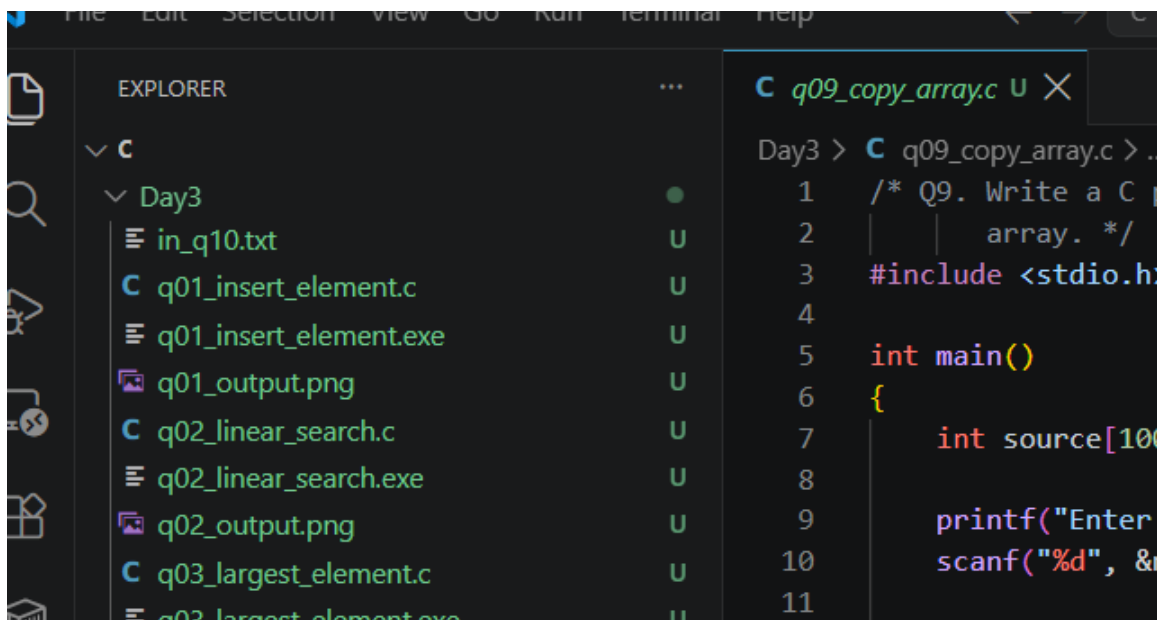
        printf("%d found at index %d\n", value, i);
        found = 1;
        break;
    }
}

if (found == 0)
    printf("Not found\n");

return 0;
}

```

Output:



Question 3: Find the largest element in an array

Program:

```

/* Q3. Find the largest element in an array. */
#include <stdio.h>

int main()
{
    int arr[100], n, i, largest;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    largest = arr[0];
}

```

```

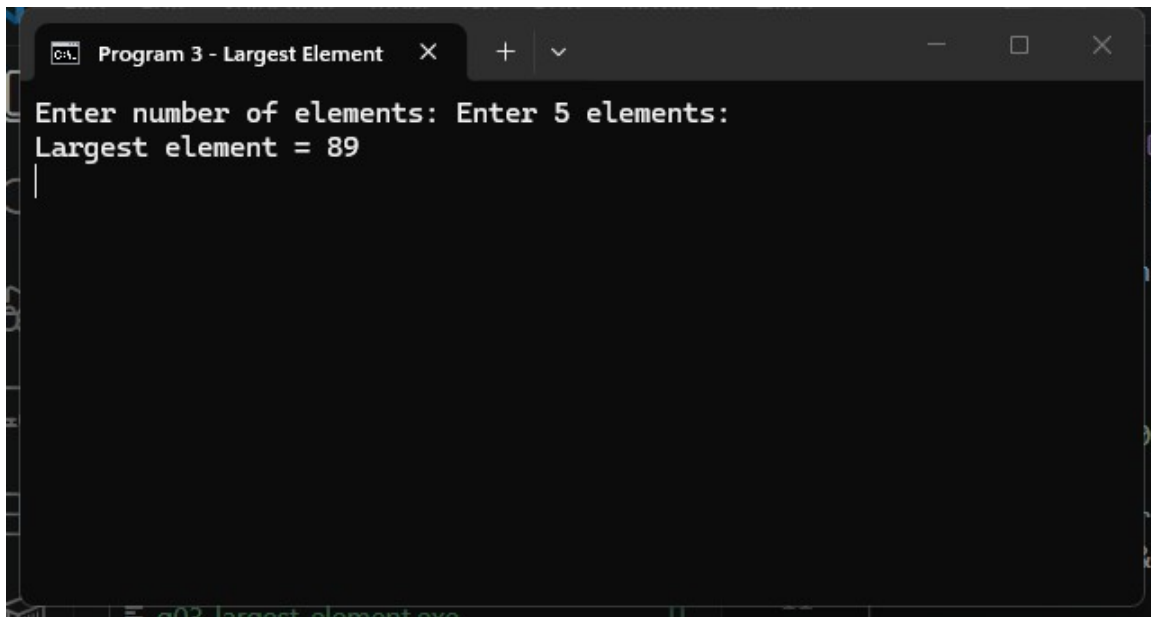
    for (i = 1; i < n; i++)
    {
        if (arr[i] > largest)
            largest = arr[i];
    }

    printf("Largest element = %d\n", largest);

    return 0;
}

```

Output:



```

Program 3 - Largest Element
Enter number of elements: Enter 5 elements:
Largest element = 89

```

Question 4: Count how many even and how many odd numbers are in an array

Program:

```

/* Q4. Count how many even and how many odd numbers are in an array. */
#include <stdio.h>

int main()
{
    int arr[100], n, i, even = 0, odd = 0;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    for (i = 0; i < n; i++)
    {

```

```

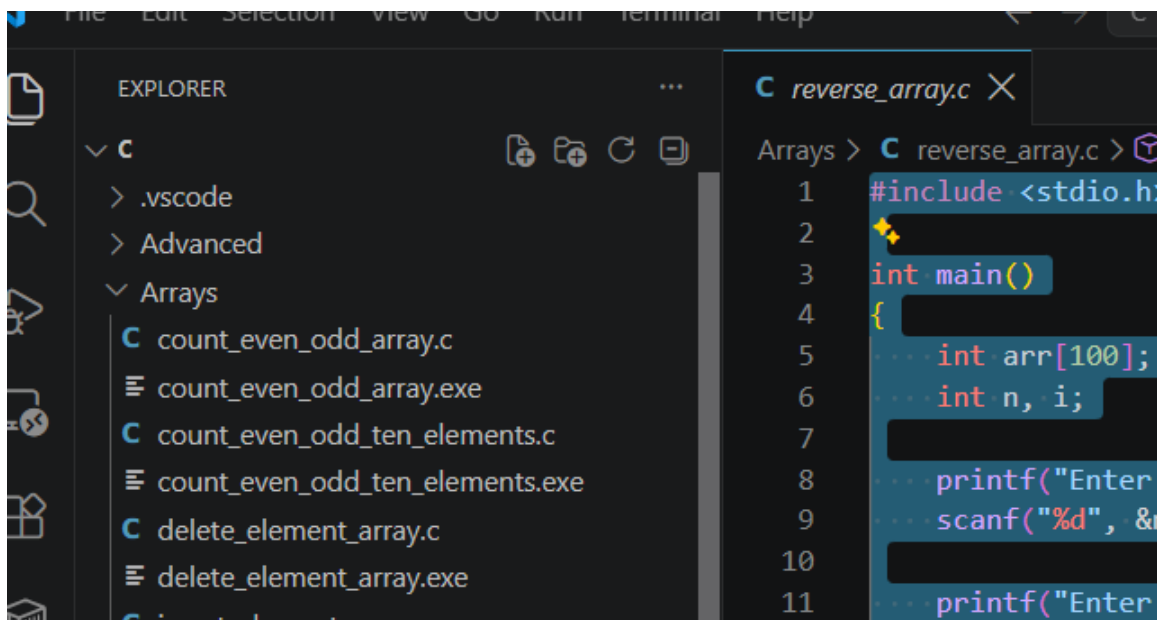
        if (arr[i] % 2 == 0)
            even++;
        else
            odd++;
    }

    printf("Even numbers: %d\n", even);
    printf("Odd numbers: %d\n", odd);

    return 0;
}

```

Output:



Question 5: Write a program to reverse an array

Program:

```

/* Q5. Write a program to reverse an array. */
#include <stdio.h>

int main()
{
    int arr[100], n, i;

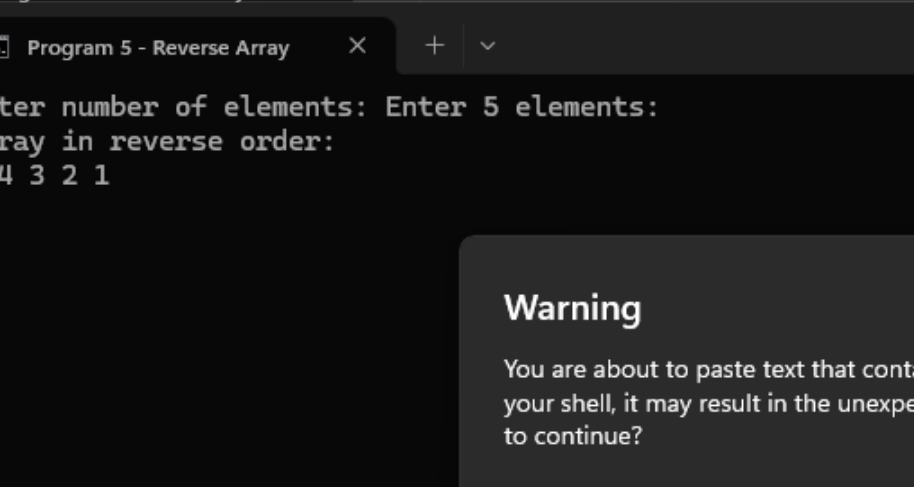
    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\\n", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("Array in reverse order:\\n");
}

```

}



```
Program 5 - Reverse Array
Enter number of elements: Enter 5 elements:
Array in reverse order:
5 4 3 2 1
```

Warning

You are about to paste text that contains multiple lines. If you paste this into your shell, it may result in the unexpected execution of commands. Do you want to continue?

Clipboard contents (preview):

```
/* Q8. Write a C program to input two
```

```
/* Q6. Delete a given value from an array. */
#include <stdio.h>

int main()
{
    int arr[100], n, i, value, position = -1;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("Enter the value to delete: ");
    scanf("%d", &value);

    for (i = 0; i < n; i++)
    {
        if (arr[i] == value)
```

```

    {
        position = i;
        break;
    }
}

if (position == -1)
{
    printf("Value not found in array\n");
    return 1;
}

for (i = position; i < n - 1; i++)
    arr[i] = arr[i + 1];

n--;

printf("Array after deletion:\n");
for (i = 0; i < n; i++)
    printf("%d ", arr[i]);
printf("\n");

return 0;
}

```

Output:

The image shows two screenshots of a C program execution. The top screenshot shows the program running with input '5' for the number of elements and '40' for the value to delete. The output shows the array '10 20 40 50' after deletion. The bottom screenshot shows the same program running with input '5' for the number of elements and '50' for the value to delete. The output shows the array '10 20 40 50' after deletion.

Question 7: Swap first and last elements in an array by getting user input

Program:

```

/* Q7. Write a C program to swap first and last elements in an array
   by getting user input. */

```

```

#include <stdio.h>

int main()
{
    int arr[100], n, i, temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("Original array:\n");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");

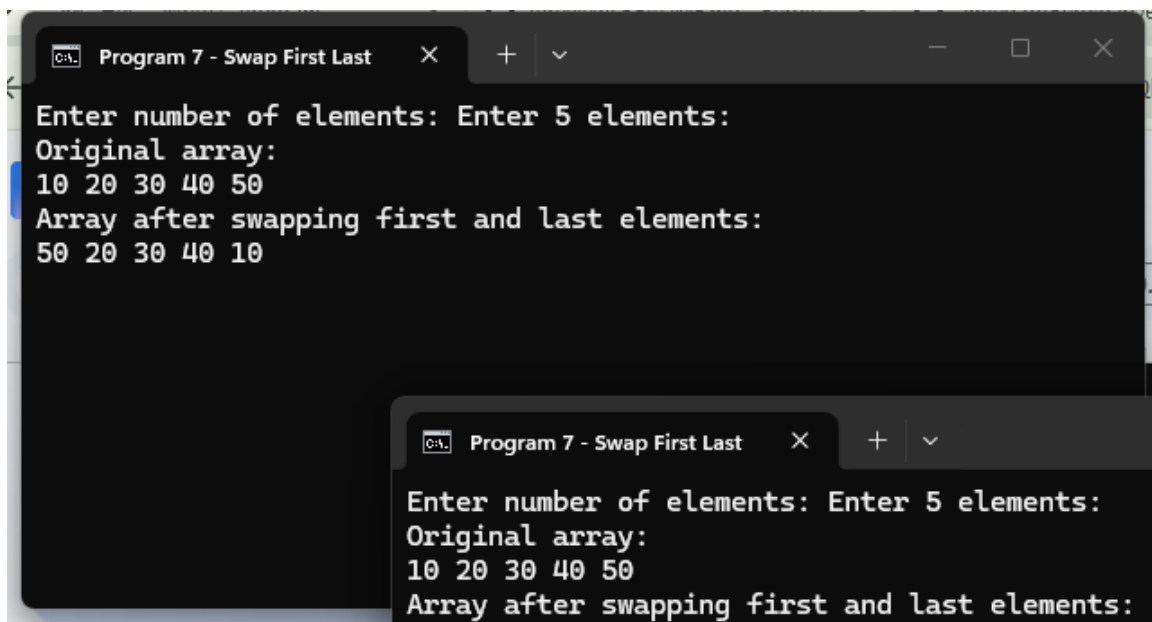
    temp = arr[0];
    arr[0] = arr[n - 1];
    arr[n - 1] = temp;

    printf("Array after swapping first and last elements:\n");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");

    return 0;
}

```

Output:



```

Program 7 - Swap First Last
Enter number of elements: Enter 5 elements:
Original array:
10 20 30 40 50
Array after swapping first and last elements:
50 20 30 40 10

```


Question 8: Input two arrays and check whether they are identical

Program:

```
/* Q8. Write a C program to input two arrays and check whether they
   are identical. */
#include <stdio.h>

int main()
{
    int a[100], b[100], n, i, identical = 1;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements of first array:\n", n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

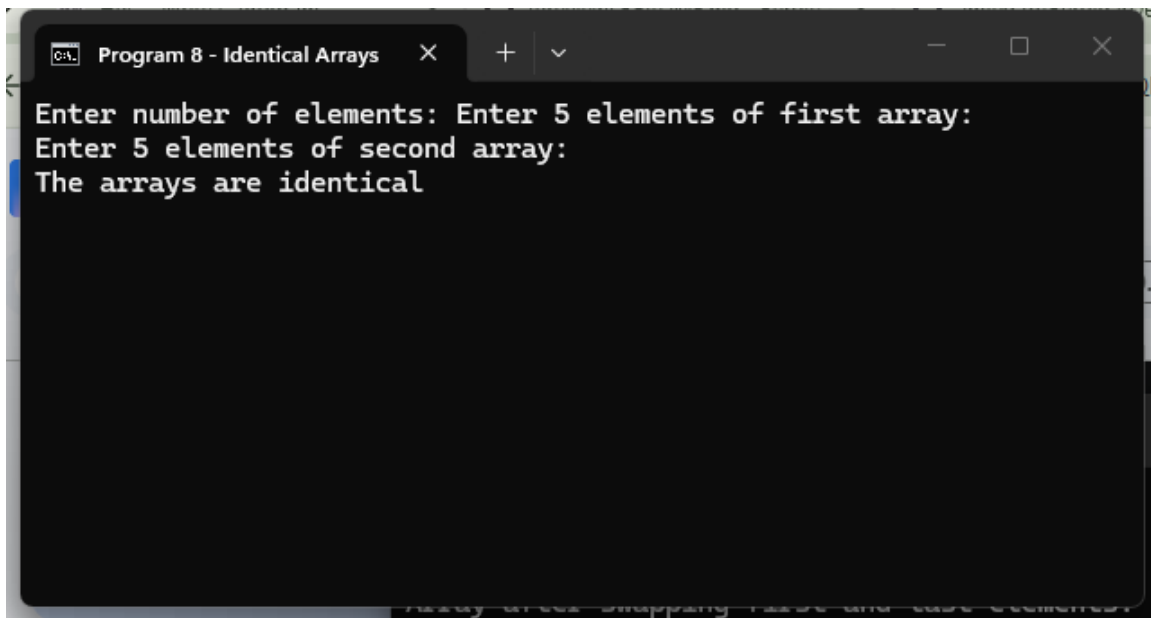
    printf("Enter %d elements of second array:\n", n);
    for (i = 0; i < n; i++)
        scanf("%d", &b[i]);

    for (i = 0; i < n; i++)
    {
        if (a[i] != b[i])
        {
            identical = 0;
            break;
        }
    }

    if (identical == 1)
        printf("The arrays are identical\n");
    else
        printf("The arrays are not identical\n");

    return 0;
}
```

Output:



Question 9: Copy all elements of one array into another array

Program:

```
#include <stdio.h>

int main()
{
    int source[100], copy[100], n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

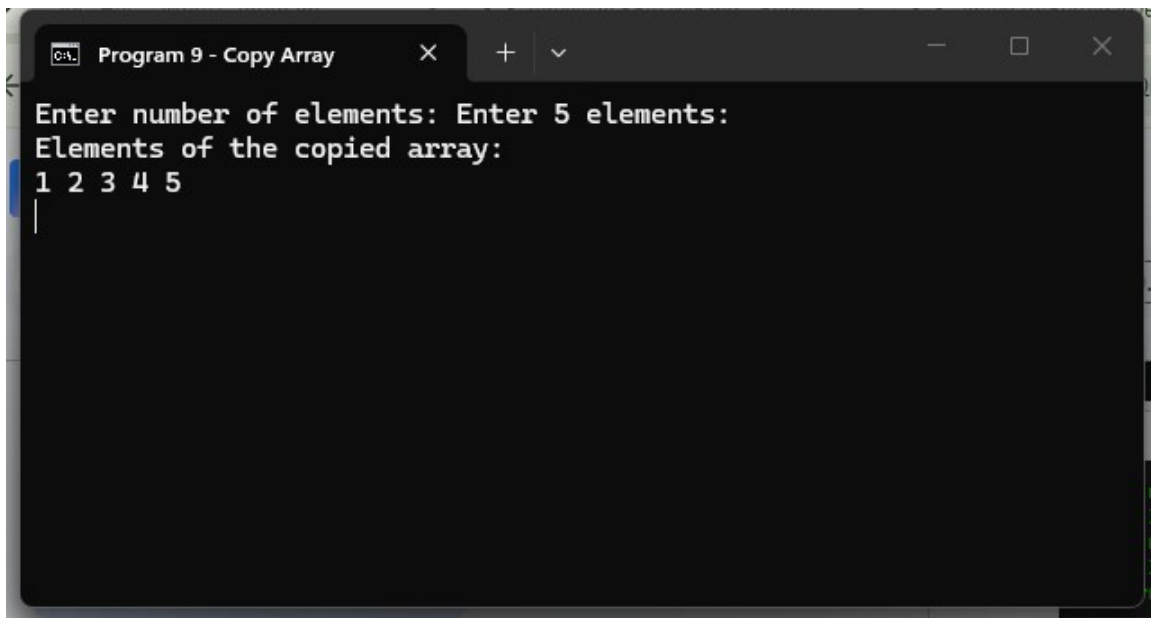
    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++)
        scanf("%d", &source[i]);

    for (i = 0; i < n; i++)
        copy[i] = source[i];

    printf("Elements of the copied array:\n");
    for (i = 0; i < n; i++)
        printf("%d ", copy[i]);
    printf("\n");

    return 0;
}
```

Output:



```
C:\> Program 9 - Copy Array
Enter number of elements: Enter 5 elements:
Elements of the copied array:
1 2 3 4 5
```

Question 10: Replace every negative number in an array with 0 and display the updated array

Program:

```
/* Q10. Write a C program to replace every negative number in an array
   with 0 and display the updated array. */
#include <stdio.h>

int main()
{
    int arr[100], n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("Original array:\n");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");

    for (i = 0; i < n; i++)
    {
        if (arr[i] < 0)
            arr[i] = 0;
    }

    printf("Array after replacing negatives with 0:\n");
    for (i = 0; i < n; i++)
```

```
    printf("%d ", arr[i]);  
    printf("\n");  
  
    return 0;  
}
```

Output:

```
Program 10 - Replace Negativ X + v  
Enter number of elements: Enter 5 elements:  
Original array:  
10 -5 0 -8 15  
Array after replacing negatives with 0:  
10 0 0 0 15
```

Section B: MCQ Answers

MCQ 1

What will be the output of the following code? `int arr[5] = {1, 2, 3}; printf("%d", arr[4]);`

Garbage value

3

0

Compilation error

Answer: c) 0

Explanation: The remaining elements of a partially initialized array are set to 0.

MCQ 2

Which statement correctly finds the number of elements in an array `arr`?

`length(arr)`

arr.length

sizeof(arr) / sizeof(arr[0])

count(arr)

Answer: c) sizeof(arr) / sizeof(arr[0])

Explanation: sizeof(arr) gives total bytes, sizeof(arr[0]) gives bytes of one element.

MCQ 3

What is the output of this code? `int arr[3][2] = {{1, 2}, {3, 4}, {5, 6}}; printf("%d", arr[2][1]);`

5

6

4

2

Answer: b) 6

Explanation: Row index 2, column index 1 points to the element 6.

MCQ 4

In C, local arrays declared inside a function without initialization contain:

Always 0

Always 1

Garbage (indeterminate) values

NULL

Answer: c) Garbage (indeterminate) values

Explanation: Automatic (local) arrays are not initialized by default.

MCQ 5

What does the following code print? `int arr[] = {10, 20, 30, 40, 50}; for (int i = 4; i >= 0; i--) printf("%d ", arr[i]);`

10 20 30 40 50

50 40 30 20 10

Compilation error

40 30 20 10 0

Answer: b) 50 40 30 20 10

Explanation: The loop prints the array from the last index down to 0.

MCQ 6

Which of these correctly deletes the element at index 2 from an array (shifting the rest left)? `int arr[5] = {10, 20, 30, 40, 50}; int size = 5;`

`arr[2] = 0;`

`for(int i = 2; i < size-1; i++) arr[i] = arr[i+1]; size--;`

`arr[2] = arr[size-1]; size--;`

`free(arr[2]);`

Answer: b) `for(int i = 2; i < size-1; i++) arr[i] = arr[i+1]; size--;`

Explanation: Shifting elements left from the deleted index and reducing size.

MCQ 7

What is the output? `int arr[4] = {5, 10, 15, 20}; int sum = 0; for (int i = 0; i < 4; i++) { sum += arr[i]; if (i == 2) break; } printf("%d", sum);`

50

30

15

45

Answer: b) 30

Explanation: $sum = 5 + 10 + 15 = 30$, then the loop breaks at $i == 2$.

MCQ 8

Given this partially filled 2D array, what is the output? `int arr[3][3] = { {1, 2}, {3}, {4, 5, 6} }; printf("%d %d %d", arr[0][2], arr[1][1], arr[1][2]);`

Garbage garbage garbage

0 0 0

Compilation error

2 3 0

Answer: b) 0 0 0

Explanation: Missing elements of a partially initialized array are set to 0.

MCQ 9

What is the output of this code? `int arr[5] = {1, 2, 3, 4, 5}; for (int i = 0; i < 5; i++) { arr[i] = arr[4 - i]; } printf("%d %d %d", arr[0], arr[2], arr[4]);`

5 3 1

5 3 5

1 3 5

5 3 3

Answer: b) 5 3 5

Explanation: After the loop the array becomes {5, 4, 3, 4, 5}, so `arr[0]=5`, `arr[2]=3`, `arr[4]=5`.

MCQ 10

What is the output of this code? `int arr[2][2][2] = {1, 2, 3, 4, 5, 6, 7, 8}; printf("%d", arr[1][0][1]);`

4

6

5

7

Answer: b) 6

Explanation: Flat index of `arr[1][0][1] = 1*4 + 0*2 + 1 = 5`, which is the value 6.

Section C: Explain

Question 1

Differentiate between compile-time initialization and run-time initialization of arrays, with examples of each.

Answer: Compile-time initialization: the array is given values at the time of declaration, before the program runs. For example: `int arr[5] = {10, 20, 30, 40, 50};` Here memory is allocated and filled when the program is compiled. If fewer initializers are given than the size, the remaining elements are automatically set to 0.

Run-time initialization: values are assigned while the program is executing. For example, reading values from the user: `for(i = 0; i < n; i++) scanf("%d", &arr[i]);` or by assignment statements such as `arr[0] = 10;` after the array has been declared.

Question 2

How is the length of an array calculated in C, given that C has no built-in length function? Explain with the underlying logic.

Answer: The length is calculated as `sizeof(arr) / sizeof(arr[0])`. `sizeof(arr)` returns the total number of bytes occupied by the whole array, and `sizeof(arr[0])` returns the number of bytes of one element. Dividing the total bytes by the bytes of one element gives the number of elements. Example: `int a[10];` `length = sizeof(a) / sizeof(a[0]) = 40 / 4 = 10`. This works only in the scope where the full array declaration is visible; inside a function parameter it decays to a pointer, so `sizeof` would return the pointer size instead.

Question 3

Explain how a 2D array is stored in memory, and how the compiler computes the address of a specific element `arr[i][j]`.

Answer: A 2D array is stored in row-major order, i.e. all the elements of row 0 are stored in consecutive memory locations, followed by all the elements of row 1, and so on. If the base address of the array is `B` and each element takes `sizeof(type)` bytes, the compiler computes the address of element `arr[i][j]` as:
`Address = B + (i * number_of_columns + j) * sizeof(type)`. Example: for `int a[3][4]` the address of `a[2][1]` is `B + (2 * 4 + 1) * 4 = B + 36` bytes.

Question 4

Write the syntax for declaring and initializing the following: (a) A 1D array (b) A 2D array (c) A 3D array.

Answer: (a) 1D array: `int a[5] = {1, 2, 3, 4, 5};`
(b) 2D array: `int a[2][3] = {{1, 2, 3}, {4, 5, 6}};`
(c) 3D array: `int a[2][2][2] = {{{1, 2}, {3, 4}}, {{5, 6}, {7, 8}}};`

Question 5

Write the output of the following programs: (a) 1D: `int a[] = {10, 20, 30, 40}; printf("%d %d", a[1], a[3]);`
(b) 2D: `int a[2][2] = {{1, 2}, {3, 4}}; printf("%d %d", a[0][1], a[1][0]);` (c) 3D: `int a[2][2][2] = {{{1,2},{3,4}}, {{5,6},{7,8}}}; printf("%d %d", a[0][1][1], a[1][0][0]);`

Answer: (a) Output: 20 40 (`a[1] = 20` and `a[3] = 40`)
(b) Output: 2 3 (`a[0][1] = 2` and `a[1][0] = 3`)
(c) Output: 4 5 (`a[0][1][1] = 4` and `a[1][0][0] = 5`)

Question 6

What happens if you try to access an element outside the array bounds?

Answer: C does not perform bounds checking. Accessing an element outside the valid range (0 to size-1) is undefined behaviour. The program may read garbage values, overwrite unrelated memory, crash, or appear to run normally. Always use valid indices within 0 to n-1.

Question 7

Find the Error: `#include <stdio.h> int main() { int a[3] = {10, 20, 30}; printf("%d", a[3]); return 0; }` 1. Is there any error in this program? 2. Explain why it occurs. 3. Write the corrected statement.

Answer: 1. Yes, the program has an error.

2. The array `a` has size 3, so valid indices are 0, 1 and 2 only. `a[3]` is out of bounds; accessing it is undefined behaviour and may print a garbage value or crash.

3. Corrected statement: `printf("%d", a[2]);` (which prints the last valid element 30)

Question 8

What is wrong with this program? `int a[5]; for(int i = 0; i <= 5; i++) { scanf("%d", &a[i]); }` Identify the mistake. What will happen during execution? Correct the program.

Answer: Mistake: the loop runs for `i = 0` to `5`, so it writes into `a[5]`, which is outside the array (valid indices are 0 to 4).

During execution: the program writes one element beyond the array, corrupting adjacent memory. It is undefined behaviour and may crash or produce incorrect results.

Corrected program: `for(int i = 0; i < 5; i++) { scanf("%d", &a[i]); }`

Question 9

Find the Output: `int a[] = {2, 4, 6, 8, 10}; for(int i = 0; i < 5; i++) { if(i % 2 == 0) printf("%d ", a[i]); }`

Answer: The condition `i % 2 == 0` is true for `i = 0, 2, 4`. Those indices hold 2, 6 and 10. Output: 2 6 10

Question 10

What is the Output? `int a[] = {1, 2, 3, 4, 5}; for(int i = 0; i < 5; i++) { a[i] = a[i] * 2; } printf("%d", a[3]);`

Answer: Every element is doubled, so `a[3]` becomes $4 * 2 = 8$. Output: 8